# Wit

### Write everything. Render anything.

A prose-first markup language for people who write — and the systems that consume what they write.

---

## What Wit is

Wit is a plain-text markup language with the formatting power of HTML and the composability of a component framework, designed around a single principle: **the writer should never have to take off the writing hat.**

You write prose. The structure, the formatting, the citations, the figures, the branching, the data — all of it lives in a syntax light enough that it never interrupts the sentence you're in the middle of. And because a Wit document parses into a clean tree — the same kind of structure HTML produces — anything downstream can consume it: a renderer, a typesetter, a game engine, a static-site builder, a data pipeline.

It is one source format for books, theses, academic articles, blog posts, reports, technical documentation, and branching interactive scripts. The writer writes once. The renderer decides what it becomes.

---

## Why Wit exists

Markdown is wonderful until the moment your prose stops being filler around code and becomes the thing itself. Then it turns on you.

Markdown's default is **markup unless escaped.** Every line you type is treated as potential structure until proven otherwise. For a README that's fine — the prose is incidental. For a novel, an essay, or a paper, it's a quiet disaster:

- A sentence beginning "*1970. It was a cold winter.*" becomes a numbered list, and your first word vanishes into a bullet.
- A line beginning "*> half of them never wrote back*" becomes a blockquote, because you happened to start with "greater than."

- Your line breaks evaporate, because single newlines collapse to spaces, and the only way to keep one is two **invisible** trailing spaces that editors strip.
- "*I multiplied 5*6*7*" renders with the 6 in italics.
- A short line above a `---` divider silently becomes a giant heading.
- The moment you need a callout, an aside, or a figure with a caption, there's no syntax — so you drop out of your manuscript and start typing raw `<figure><figcaption>` HTML.
- References are the deepest cut: you number footnotes by hand, you maintain a bibliography linked to nothing, and "see Figure 3" is a magic constant that becomes a lie the moment you reorder anything.

Every one of those failures is the same bug wearing a different hat: **markdown reads as control characters the things that are, in prose, just content** — the first character of a line, ordinary punctuation, whitespace. And the betrayal is invisible in the source. Everything looks correct while you type it; you find out it broke only when you render.

Wit inverts the default. **Prose unless marked.** A blank line is a paragraph. A sentence is a sentence. Nothing you type means anything special until you deliberately, visibly say so. The writer stays in flow. The structure is opt-in, never ambient.

---

## The core ideas

**1. Two audiences, one file.** A writer opens a Wit file and sees prose with light annotations. A programmer opens the same file and sees a component tree with data, control flow, and scripting. Neither is wrong. Neither gets in the other's way. The base language is complete for someone who will never write a single piece of logic — and the moment a programmer needs power, it's there, off to the side, where the prose stays clean.

**2. `@` uses, `#` defines.** One rule, no exceptions. You *use* a node with `@`. You *define* one with `#`. Everything in the language is one of those two gestures.

**3. No fixed vocabulary.** HTML gives you a finite set of tags decided decades ago — `<div>`, `<p>`, `<section>` — and the instant you need something it didn't anticipate, you're reaching for `class="..."` and hoping the meaning lands somewhere. Wit has no built-in vocabulary you're stuck with. A novelist defines `@chapter`, `@thought`, `@letter`. An academic defines `@theorem`, `@finding`, `@proof`. A game writer defines

`@keeper`, `@player`, `@choice`. The node name *is* the meaning, and the renderer decides what each name becomes.

**4. It's a DOM.** A Wit document parses into a tree, exactly like HTML or XML. Node names carry semantics, nesting carries hierarchy, parameters carry attributes. Downstream systems walk that tree the way they'd walk a DOM — which means consuming a Wit document is a solved problem, not a parsing adventure.

**5. The writer expresses intent. The renderer supplies meaning.** This is the whole thesis. Wit is a semantic intention layer that sits between human writing and whatever system needs it. You say *what you mean*. The renderer decides what that means in context — a book, a website, a slide deck, a dialogue tree.

---

# The basics

## Prose

Prose is the default. Write naturally. A blank line separates paragraphs. Nothing is marked up.

```
The keeper had not spoken aloud in eleven days.

He had not noticed, until his own voice surprised him.
```

## Emphasis

Two inline marks, and only two:

```
This is _italic_ and this is *bold*.
```

`_underscore_` is italic — the old typewriter convention where underlining meant "set this in italics." `*asterisk*` is bold — the heavier, louder character earns the heavier weight. These are the only inline marks in the base language, and because they wrap a token rather than living at the start of a line, ordinary punctuation in your prose can never trigger them by accident.

## Comments

Anything between `\-` and `-\` is a comment. It never renders — invisible stage directions for the writer.

```
\- remember to verify this date before publishing -\
The lighthouse was commissioned in 1847.
```

---

# Nodes

A node is the unit of structure. It has a name, an optional set of parameters, and an optional body.

## Using a node

Open with `@name`, close with `name@`. Everything between is the body.

```
@aside
Fresnel lenses rarely fail. Keepers do.
aside@
```

Inline works the same way:

```
She crossed the @highlight last threshold highlight@ alone.
```

Whether `@aside` renders as a block and `@highlight` renders inline is a decision the *renderer* makes based on the node name — not something the writer declares. The writer just wraps content in named nodes and the renderer knows what each one means.

## Parameters

Parameters live inside pipes and can appear **anywhere** inside a node — before the body, after it, mid-sentence. The parser collects them regardless of position, so the writer places them wherever the thought lands.

```
@figure
|lamp.png|
|caption - The second-order Fresnel lens|
|full width|
figure@
```

There are three forms:

- `|value|` — a positional value.
- `|key - value|` — a named value. Everything left of the first `-` is the key; everything right of it is the value. Whitespace is allowed on both sides, so `| background colour - dark slate|` reads as naturally as a label on an object.
- `|flag|` — a boolean flag. No hyphen means the value is simply *present*.

If the same parameter appears more than once, the **last one wins**, applied from that point forward in the flow. Changing your mind mid-node is just a matter of dropping a new pipe:

```
@scene
|mood - calm|
The harbour was still.
|mood - tense|
Then the bell began to ring.
scene@
```

## Indentation is cosmetic

Scope comes entirely from the `@name` / `name@` pairs — never from whitespace. Indent for your own readability, or don't. Both parse identically. A writer who finds indentation fussy ignores it completely; a programmer who wants visual structure uses it freely. Neither is wrong.

```
@chapter
@aside Keepers do. aside@
He read the letter twice.
chapter@
```

is identical to:

```
@chapter
    @aside Keepers do. aside@
    He read the letter twice.
chapter@
```

---

# Defining your own nodes

`#` defines. This is how you build the vocabulary for your document.

## A node with a body

`#name` opens the definition, `name#` closes it. Inside:

- `||param||` **captures** a parameter the node accepts. List several with `||a, b, c||`.
- `::param::` **interpolates** a captured parameter's value.
- `...` marks where the body of each invocation will render.

```
#chapter
||number, title, subtitle||
@chapterheader
::number:: — ::title::
(if ::subtitle::)
::subtitle::
(end)
chapterheader@
...
chapter#
```

Used like this:

```
@chapter
|number - I|
|title - The Keeper|
|subtitle - On the nature of attention|

For eleven days the lamp had burned untended.
chapter@
```

The captured parameters flow into the definition's structure, and the body prose lands wherever `...` sits. Parameters can even be threaded into child nodes — a value passed at the use site flows down into a nested node's own parameter slot, invisibly.

### A node with no body

When something is just a named value, use the single-line form. The `:` signals "no body."

```
#year: 1923
#place: Dunmore Head
```

Single-line definitions can still capture and interpolate parameters, and complex values are wrapped in `! ... !`, which may span multiple lines and contain other nodes:

```
#cite: ||author, title, year|| ! ::author:: (::year::). _::title::_. !
```

---

## References and citations

References are where every other tool makes the writer do the machine's job. Wit doesn't.

A bare `@handle` points at something you've defined. The renderer computes any number — footnote order, citation format, figure number — so you never type a number that can drift out of sync.

Define a citation schema once:

```
#cite: ||author, title, year, publisher, page||
! ::author:: (::year::, p. ::page::). _::title::_. ::publisher::. !
```

Define your sources as named instances of it:

```
#weil:   ! Simone Weil, Gravity and Grace, 1952, Routledge, !
#berger: ! John Berger, Ways of Seeing, 1972, Penguin, !
```

Then cite in prose. The only thing you carry at the point of writing is the locator:

```
Attention, as @weil! p. 42 ! argued, is the rarest form of generosity.
```

Values for a `#thing:` call go between `! ... !`. When a reference needs no parameters at all, the bare name is enough — `@weil`.

The most powerful pattern is the **argument map**: pre-load your sources *and your ideas* at the top of a document, then write entirely in terms of them.

```
#weil_attention:  @weil! p. 42 !
#weil_generosity: @weil! p. 117 !
#berger_looking:  @berger! p. 9 !
```

Now your in-text citations are two words long, and they name *ideas*, not pages:

```
Attention, as @weil_attention argued, is the rarest form of
generosity. Berger complicates this @berger_looking by insisting
that we never look at one thing alone.
```

No citation manager does that. You're not citing a page — you're naming the intellectual landmark you're standing on, and the bibliography assembles itself from whatever you actually used.

---

# Data

When a document needs structured data — characters, datasets, configuration, records — Wit holds it statically. Nothing is ever created or mutated as the document flows; data is defined once and referenced anywhere.

A writer never has to touch this. By the time you reach for it, you're thinking like a programmer, so the syntax leans on conventions programmers already know.

## Records

A record is a set of named values wrapped in `{ }`. Keys use the same `-` delimiter as parameters. The opening brace sits on the definition line, and the body is indented — never a brace stranded alone on its own line.

```
#keeper: {
  name - Aldous Vane
  years at post - 31
  lamp lit - true
}
```

Short records fit on one line, comma-separated:

```
#world: { location - Bag End, time - night, storm - true }
```

Records nest:

```
#keeper: {
  name - Aldous Vane
  posted - Dunmore Head
  history { years - 31, incidents - 2 }
}
```

## Collections

`[ ]` holds a collection — of plain values, or of records. The opening bracket sits on the definition line; each record is indented beneath it.

A collection of values:

```
#themes: [ attention, perception, moral failure ]
```

A collection of records — the common case for tables, casts, datasets:

```
#findings: [
  { claim - Attention precedes perception, supported - true,  strength - strong   }
  { claim - Looking is never neutral,      supported - true,  strength - strong   }
  { claim - Attention is a form of labour, supported - true,  strength - moderate }
  { claim - Perception is neutral,         supported - false, strength - contested }
]
```

Records inside a collection are the same `{ }` records as above — one syntax for records everywhere. They map directly to the JSON-shaped data a downstream renderer expects, so there is nothing to translate.

## Accessing data

Dot notation reaches into records and collections:

```
@keeper.name served for @keeper.years_at_post years.
@findings.0.claim
```

The renderer fuzzy-matches keys, so `years at post` and `years_at_post` resolve to the same thing — the writer never has to remember exact key formatting.

---

## Control flow

Wit has no variables and no computation in the document body — but it does have declarative control flow, the same shape as a template engine. The only things a condition or loop can reference are already-defined static data. Statements are wrapped in parentheses, so logic reads as an aside, never an instruction.

### Conditionals

```
(if @book.status = draft)
@watermark Draft — do not distribute. watermark@
(end)
```

```
(if @author.corresponding = true)
Corresponding author: @author.email
(else)
@author.name
(end)
```

### Iteration

```
(each @findings as finding)
@finding
|claim - @finding.claim|
|strength - @finding.strength|
finding@
(end)
```

`(end)` closes the nearest open statement. Statements nest freely. Because the loop walks static data, there is no iteration state, no mutation, no surprises — just a clean expansion of the tree.

This is what makes a Wit document *self-organising*. Define your chapters as data, and both your table of contents and your chapter bodies can render from the same

source. Add a record; both update. Reorder them; both follow. Nobody maintains anything by hand.

---

## Composing across files

A book, a thesis, or a large report should not live in one file. Wit composes.

### Referencing

At the top of a file, `reference` pulls in the definitions, data, and nodes from another file.

```
reference ./shared/schema.wit
reference ./shared/sources.wit
reference ./chapters/one.wit
reference ./chapters/two.wit
```

A master file becomes a composition script: it assembles the pieces. Each chapter file is self-contained prose that reaches into the shared vocabulary.

### Additive partials

Prefix a definition with `+` and multiple declarations of the same name **merge** instead of overwriting. A shared collection that many files contribute to.

```
\- in chapters/one.wit -\
+#bibliography:
@weil:   ! Simone Weil, Gravity and Grace, 1952, Routledge !
@berger: ! John Berger, Ways of Seeing, 1972, Penguin !

\- in chapters/two.wit -\
+#bibliography:
@arendt: ! Hannah Arendt, The Human Condition, 1958, Chicago !
```

The renderer sees one merged `@bibliography` containing every contribution. No file knows about the others.

This makes a project genuinely self-assembling. Each chapter file registers its own table-of-contents entry and its own sources:

```
\- chapters/three.wit -\
+#toc:
@tocrow |number - III| |title - The Politics of Looking| tocrow@

+#bibliography:
@berger_power: @berger! p. 86 !
```

The master file has *no chapter list*. It just references the files and renders `@toc` and `@bibliography` — both fully populated by the chapters that registered themselves. Add a chapter: create the file, add one `reference` line. The contents and bibliography grow automatically. Delete a chapter: remove the line, and it vanishes from both. The document reorganises itself.

Additive partials are for things that accumulate — bibliographies, glossaries, indexes, figure lists, tables of contents. They are not for structural nodes whose bodies can't be meaningfully merged.

---

## Scripting

The escape hatch for everything the declarative model can't express is plain JavaScript, in a `<script>` tag — exactly like HTML. Wit does not reinvent the wheel.

A script runs **once, after the initial DOM is rendered**. It reads the parsed tree through a small bridge object, `lh`, manipulates it, and the final DOM passes to the next system. (When the runtime is later persisted, the same script tag becomes a live loop — the document becomes reactive without a framework living inside the language.)

```
<script>
// read the data
const findings = lh.data.findings;
const supported = findings.filter(f => f.supported === 'true').length;
const pct = Math.round((supported / findings.length) * 100);

// compute reading time from actual prose
const words = lh.prose().wordCount();
lh.set('paper.word count', words);

// sort findings by strength before the final render
```

```
const order = { strong: 0, moderate: 1, contested: 2 };
lh.sort('finding', (a, b) => order[a.params.strength] - order[b.params.strength]);

// inject computed content into a waiting node
lh.inject('paper-stats', `
  @statrow |label - Supported| |value - ${supported} of ${findings.length} (${pct}%)|
statrow@
  @statrow |label - Reading time| |value - ~${Math.ceil(words / 200)} min| statrow@
`);
</script>
```

The `lh` bridge exposes the document as a manipulable tree:

| Call | What it does |
|---|---|
| `lh.data` | all `#thing:` definitions as plain JavaScript objects |
| `lh.query(name)` | every node of a given type, as `{ params, content }` objects |
| `lh.node(id)` | a single node by its `id` parameter |
| `lh.sort(name, fn)` | reorder all instances of a node type |
| `lh.inject(id, source)` | render Wit source into a node by id |
| `lh.set(path, value)` | update a value in the data, e.g. `'paper.word count'` |
| `lh.prose()` | the prose text nodes, with helpers like `.wordCount()` |

The writer and the programmer work in the same file. The writer writes prose. The programmer drops a script tag at the bottom. Neither disturbs the other.

---

## The DOM, and what comes after

Everything above produces a single thing: a tree.

```
document
├── @book (title, author, status)
│   ├── @titlepage
│   ├── @toc
│   │   ├── @tocrow (number: I, title: Introduction)
│   │   └── @tocrow (number: II, title: Attention as Labour)
│   ├── @chapter (number: I)
```

```
|   |   ├── prose
|   |   └── @note
|   └── @bibliography
|       ├── @cite (author: Weil, …)
|       └── @cite (author: Berger, …)
```

That tree is the entire deliverable. Node names are element types. Parameters are attributes. Nesting is the hierarchy. Body text is content. It is, structurally, a DOM — which means any downstream system already knows how to consume it:

- A **React renderer** maps each node name to a component and walks the tree.
- A **typesetter** maps nodes to layout primitives and produces a PDF.
- A **static-site builder** maps nodes to HTML.
- A **game engine** reads the dialogue tree and steps through it at runtime.
- A **data pipeline** queries the tree for the records and collections it needs.

The same Wit source can drive all of them. The language stays neutral about output; the renderer on the other end decides whether the document becomes a book, a website, a game, a slide deck, or something that doesn't exist yet.

---

## What Wit is for

Any document where a great deal of formatting is needed, but the act of managing that formatting is destructive to the prose the writer is trying to produce:

- **Books and manuscripts** — chapters across files, self-assembling front matter, a vocabulary the author defines once and writes inside of.
- **Academic articles and theses** — citations that read as ideas, footnotes and figure numbers computed automatically, a bibliography that builds itself from what was actually cited.
- **Blog posts and essays** — prose-first writing with callouts, asides, and figures that never force the writer into raw HTML.
- **Reports** — data defined once, driving both the prose and the tables, with conditional sections that appear based on status, audience, or review state.
- **Interactive and branching scripts** — RPG dialogue trees written as nested prose, where the conversation hierarchy *is* the document structure and the branches gate on game state.

- **Technical documentation** — components, reusable definitions, data-driven reference tables, and scripted computation, all in a source that still reads as writing.

---

## Worked examples

Six documents, six genres, one language. Each leans on a different part of Wit, but all of them stay in prose mode and all of them parse to the same kind of tree.

### A report

Reports live or die on structure that reflects data: metrics measured against targets, status that changes the document, sections that appear or vanish depending on whether the thing is a draft or a final. Define the data once; let it drive the prose, the tables, and the conditional sections.

```
\- quarterly-report.wit -\

#report: {
  title - Q3 Operations Review
  author - Mara Finch
  department - Coastal Infrastructure
  quarter - Q3 2024
  status - final
}

#metrics: [
  { label - Uptime,        value - 99.2%,  target - 99.5%, met - false }
  { label - Incidents,     value - 3,      target - 5,     met - true  }
  { label - Response time, value - 12 min, target - 15 min, met - true }
  { label - Budget used,   value - 84%,    target - 100%,  met - true  }
]

#sites: [
  { name - Dunmore Head,  status - operational, inspected - 2024-08-14 }
  { name - Carrick Point, status - maintenance, inspected - 2024-09-02 }
  { name - Ines Rock,     status - operational, inspected - 2024-07-30 }
]
```

```
\- a metric renders green when it hit target, amber when it didn't -\
#metric ||label, value, target, met||
  @metriccard
    @metriclabel ::label:: metriclabel@
    @metricvalue ::value:: metricvalue@
    (if ::met:: = true)
      @badge |tone - good| on target badge@
    (else)
      @badge |tone - warn| below target — goal ::target:: badge@
    (end)
  metriccard@
metric#

@document |title - @report.title|

  (if @report.status = draft)
    @watermark Draft — not for circulation watermark@
  (end)

  @reportheader
    @report.title
    @report.department · @report.quarter
    Prepared by @report.author
  reportheader@

  @h1 Executive summary h1@

  Operations held steady through Q3 despite the Carrick Point outage
  in August. Three of four targets were met; uptime fell marginally
  short owing to the scheduled maintenance window. No safety
  incidents were recorded.

  @h1 Key metrics h1@

  (each @metrics as m)
    @metric |label - @m.label| |value - @m.value| |target - @m.target| |met - @m.met|
metric@
  (end)

  @h1 Site status h1@
```

```
  (each @sites as site)
    @siterow
      @sitename @site.name sitename@
      @sitestatus |state - @site.status| @site.status sitestatus@
      Last inspected @site.inspected
    siterow@
  (end)

  @h1 Recommendations h1@

  @recommendation |priority - high|
    Bring the Carrick Point lamp replacement forward to Q4 to recover
    the uptime shortfall before storm season.
  recommendation@

  @recommendation |priority - medium|
    Increase inspection frequency at Ines Rock after the long gap
    since July.
  recommendation@

document@
```

Flip `status` to `draft` and the watermark reappears. Add a row to `#metrics` and the table grows itself. The prose never mentions a number that the data doesn't already hold.

## A blog post draft

A blog post is the purest test of prose-first: it's casual, it's mostly sentences, and the few rich elements — a callout, a code sample, a pull quote, an image — must never drag the writer into HTML mid-thought. The `status - draft` flag is a single switch the writer flips before publishing.

```
\- why-i-quit-my-citation-manager.wit -\

#post: {
  title - Why I stopped using a citation manager
  author - Aldous Vane
  date - 2024-09-18
  status - draft
```

```
}

#tags: [ writing, tools, academia ]

@article |title - @post.title|

  @postmeta
    @post.author · @post.date
    (each @tags as tag)
      @tag @tag tag@
    (end)
  postmeta@

  @h1 @post.title h1@

  For three years I kept my references in a database. Every paper I
  read, I logged — author, title, year, the lot. And every time I sat
  down to actually _write_, I broke flow to hunt down a citation key,
  paste it in, and lose my thread entirely.

  @callout |tone - aside|
    The tool meant to help me write was the thing stopping me from
    writing.
  callout@

  Then I tried defining my sources once, in plain text, and citing
  them by what they _mean_ rather than by some opaque key:

  @code |language - wit|
    #weil_attention: @cite! Simone Weil, Gravity and Grace, 1952, p. 42 !

    Attention, as @weil_attention argued, is the rarest generosity.
  code@

  @pullquote
    I wasn't citing a page anymore. I was naming the idea I was
    standing on.
  pullquote@

  @figure |src - desk.jpg| |caption - My actual desk, mid-draft.|
```

```
  figure@

  @h2 The point h2@

  The friction was never the references. It was the context-switch —
  leaving the writing to manage the writing. Delete that, and you
  write faster *and* cite better.

article@
```

## An academic article

The hardest formatting burden in scholarship is the reference apparatus, and Wit dissolves it. Define a citation schema once, name your sources, then build an *argument map* — handles that name ideas, not pages. In the body, every citation is two words and reads as prose. Footnotes are inline nodes; figure numbers and the bibliography compute themselves.

```
\- attention.wit -\

#cite: ||author, title, year, page|| ! ::author:: (::year::, p. ::page::). _::title::_
. !

#weil:   ! Simone Weil, Gravity and Grace, 1952 !
#berger: ! John Berger, Ways of Seeing, 1972 !
#arendt: ! Hannah Arendt, The Human Condition, 1958 !

\- argument map: name the idea, not the page -\
#weil_attention:  @weil! 42 !
#weil_generosity: @weil! 117 !
#berger_looking:  @berger! 9 !
#arendt_labour:   @arendt! 79 !

#theorem ||label, title||
  @theoremheader ::label:: — ::title:: theoremheader@
  ...
theorem#

@article
  |title - On Attention and the Act of Looking|
```

```
|author - Aldous Vane|
|institution - University of the Coast|

@abstract
  This paper argues that the act of looking is never neutral, and
  that attention constitutes a form of ethical labour whose absence
  carries measurable consequence.
abstract@

@h1 Introduction h1@

Attention, as @weil_attention argued, is the rarest form of
generosity — not a cognitive habit but a moral orientation toward
the world. Berger complicates this @berger_looking by insisting
that we never look at one thing alone.@note
  This is not merely epistemological. Berger is making an argument
  about power — who looks, at what, and on whose terms.
note@

@h1 Attention as labour h1@

@theorem |label - Theorem 1| |title - The Asymmetry of Attention|
  If attention is a form of power, then its distribution is always
  already political @arendt_labour
theorem@

Weil's corrective @weil_generosity is to insist that genuine
attention requires the suspension of the self.

@figure |id - fig-model| |caption - The dual structure of attentive labour.|
figure@

As @fig-model shows, the structure holds across both accounts.

@h1 Conclusion h1@

The lamp burns only as long as someone tends it. Attention is not
incidental to the moral life — it is the moral life.

@bibliography bibliography@
```

```
article@
```

The numbers in the footnote, the figure reference, and the references list are all computed. Reorder anything and they renumber themselves, because you never typed a number.

## A book manuscript

A book shouldn't live in one file. The master file is a composition script that references the parts and renders `@toc` and `@bibliography` — both of which fill themselves, because each chapter file *registers itself* with additive partials. The author works only in chapter files, in prose.

```
\- master.wit -\

reference ./shared/schema.wit
reference ./chapters/one.wit
reference ./chapters/two.wit

#book: {
  title - The Keeper
  subtitle - Attention, Perception, and the Moral Life
  author - Aldous Vane
  status - draft
}

@book |title - @book.title| |status - @book.status|

  (if @book.status = draft)
    @watermark Draft — do not distribute watermark@
  (end)

  @titlepage
    @book.title
    @book.subtitle
    @book.author
  titlepage@

  \- the contents and bibliography fill themselves -\
  @toc toc@
```

```
  @chapter_one chapter_one@
  @chapter_two chapter_two@

  @bibliography bibliography@

book@
```

```
\- chapters/one.wit -\

\- this chapter registers itself in the front matter -\
+#toc: @tocrow |number - I| |title - The Lamp| tocrow@
+#bibliography: ! Simone Weil, Gravity and Grace, 1952 !

#chapter_one
  @chapter |number - I| |title - The Lamp|

  @epigraph
    The sea does not forgive forgetting.
    |source - coastal proverb|
  epigraph@

  For eleven days the lamp had burned untended, and Aldous told
  himself this was _fine_.

  @scenebreak scenebreak@

  He had not spoken aloud in so long that his own voice, when it
  came, surprised him.
  chapter@
chapter_one#
```

Add a third chapter: create `chapters/three.wit`, register its `+#toc` and `+#bibliography` lines, add one `reference` to the master. The table of contents and the bibliography grow on their own.

## A film script

Screenplays are highly structured prose — slug lines, action, character cues, parentheticals, transitions — which is exactly the kind of formatting that breaks a

writer's flow in a word processor. Define the screenplay vocabulary once; then write the scene as it reads on the page.

```
\- screenplay vocabulary -\

#scene ||location, time||
  @slugline ::location:: — ::time:: slugline@
  ...
scene#

#cue ||who||
  @character ::who:: character@
  ...
cue#

\- the scene -\

@scene |location - INT. LIGHTHOUSE — LANTERN ROOM| |time - NIGHT|

  @action
    Wind claws at the glass. ALDOUS VANE, fifties, weathered, trims
    the wick of a great lamp. A KNOCK at the hatch below. He does
    not turn.
  action@

  @cue |who - ALDOUS|
    @paren not turning paren@
    We're closed.
  cue@

  @action
    The hatch opens anyway. A STRANGER hauls themselves up, soaked
    through, cradling a tin.
  action@

  @cue |who - STRANGER|
    I brought oil.
  cue@

  @action
```

```
    For the first time, Aldous turns.
  action@

  @cue |who - ALDOUS|
    @paren quietly paren@
    ...You'd best come in, then.
  cue@

  @transition CUT TO: transition@

scene@
```

The same source can render as a properly formatted screenplay PDF, a table read, or a production database — the renderer decides. The writer just wrote the scene.

## A non-linear RPG script

This is where the node model earns its keep. A branching conversation *is* a tree, and Wit documents are trees — so the dialogue hierarchy falls out of the nesting. The writer writes character voices in prose; the branches are nested choices; the conditions gate on game state. No node IDs, no edge tables. Here it is set at Bag End, with the Ring newly revealed. (The dialogue is original, written for the world.)

```
\- bag-end.wit -\

\- characters and world state -\
#frodo:   { name - Frodo,   has ring - true, knowledge - low }
#gandalf: { name - Gandalf, mood - grave }
#world:   { location - Bag End, sauron knows - false }

\- one node for a spoken line -\
#say ||who, mood||
  @line |speaker - ::who::| |mood - ::mood::|
    ...
  line@
say#

@scene |id - bag-end-revelation|

  @say |who - Gandalf| |mood - grave|
    The fire confirms it. This is no trinket, Frodo. It is the
```

```
  Enemy's own — and while it rests in the Shire, the Shire is
  in peril.
say@

@choices

  \- branch: offer it away -\
  @choice
    @say |who - Frodo| |mood - afraid|
      Then you take it. You're wise, you're strong. Keep it hidden.
    say@
    @say |who - Gandalf| |mood - urgent|
      Do not offer it to me. I dare not so much as hold it. The Ring
      would find the good in me and turn it to ruin.
    say@
    @goto |scene - what-must-be-done|
  choice@

  \- branch: ask why -\
  @choice
    @say |who - Frodo| |mood - lost|
      But why me? I'm no one — a hobbit who's never been past Bree.
    say@
    @say |who - Gandalf| |mood - gentle|
      Small hands turn the wheel of the world while the eyes of the
      great look elsewhere. It fell to you. That is reason enough.
    say@

    @choices
      @choice
        @say |who - Frodo| |mood - resolved|
          Then I'll carry it. Wherever it must go.
        say@
        @goto |scene - the-road-begins|
      choice@

      \- only unlocks once Frodo has pieced things together -\
      (if @frodo.knowledge = high)
      @choice
        @say |who - Frodo| |mood - knowing|
```

```
            The Enemy is already stirring in the East, isn't he.
          say@
          @say |who - Gandalf| |mood - impressed|
            You see further than you let on. Yes — and we have less
            time than I had hoped.
          say@
        choice@
        (end)
      choices@
    choice@

    \- only appears once the Enemy is on the trail -\
    (if @world.sauron knows = true)
    @choice
      @say |who - Frodo| |mood - panicked|
        He knows it's here. He knows the name Baggins.
      say@
      @say |who - Gandalf| |mood - dark|
        He does. You cannot stay — not for another night.
      say@
      @goto |scene - flee-the-shire|
    choice@
    (end)

  choices@

scene@
```

The writer wrote Gandalf and Frodo as prose. The nesting made the tree. The `(if)` blocks read as stage directions — *this line only plays if*—. Set `world.sauron knows` to true and a whole branch appears. `@goto` is the only escape hatch, used when a path leaves the scene rather than continuing inside it.

---

## Design principles

1. **Prose unless marked.** The default is writing. Structure is always deliberate and always visible. Nothing the writer types means anything special by accident.

2. **One source, many outputs.** The document is a tree. The renderer decides what the tree becomes.
3. **Cost scales with rarity.** The thing you do most — write sentences — costs nothing. The rare things carry a light, opt-in mark.
4. **Two hats, never at once.** The prose writer never has to think like a programmer. The programmer never has to disturb the prose.
5. **Name the meaning.** Node names carry intent, so the source reads as the thing it is — `@keeper` reads as "the keeper speaks," `@finding` as "this is a finding," `@hero` as "this is the hero section."
6. **Defer to the renderer.** Wit expresses what the writer means. The system downstream supplies what that means in context.

---

## Syntax reference

| Syntax | Meaning |
|---|---|
| `_x_` | italic |
| `*x*` | bold |
| `\- x -\` | comment (never renders) |
| `@name … name@` | use a node |
| `@name` | use a node / reference with no parameters |
| `#name … name#` | define a node with a body |
| `#name:` | define a single-line node (no body) |
| `+#name` | additive partial — merges across declarations |
| `\| value \|` | positional parameter |
| `\| key - value \|` | named parameter (left of first `-` is the key) |
| `\| flag \|` | boolean flag |
| `\|\| a, b \|\|` | capture parameters (in a definition) |
| `::name::` | interpolate a captured parameter |
| `...` | body slot (where invocation content renders) |
| `! … !` | value(s) for a `#name:` call; may span lines |
| `@thing.field` | access a value in a record or collection |
| `{ … }` | a data record (keys use `-`); opens on the definition line |

| Syntax | Meaning |
| --- | --- |
| `[ … ]` | a collection of values or records |
| `(if cond)` `(else)` `(end)` | conditional |
| `(each @coll as item)` `(end)` | iteration |
| `reference ./path` | pull in definitions from another file |
| `<script> … </script>` | plain JavaScript, runs after initial render |

---

**Wit.** Your intent, made manifest.